

A. Use `EXTRACT(MONTH FROM order_date)` for month.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
4 # Use EXTRACT(MONTH FROM order_date) for month.
5
6 SELECT
7     EXTRACT(YEAR FROM STR_TO_DATE(order_date, '%d-%m-%Y')) AS order_year,
8     EXTRACT(MONTH FROM STR_TO_DATE(order_date, '%d-%m-%Y')) AS order_month
9 FROM ecommerce_sales;
10
11
```

The Result Grid shows the following data:

order_year	order_month
2023	12
2025	4
2024	10
2024	9
2024	12
2024	4
2025	5
2023	9
2023	10
2023	10
2024	2

The Action Output shows the execution of the query:

#	Time	Action	Message	Duration / Fetch
34	14:08:56	select * from ecommerce_sales LIMIT 0, 1000	1000 row(s) returned	0.015 sec / 0.000 sec

B. GROUP BY year/month.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
11 #GROUP BY year/month.
12 SELECT
13     YEAR(STR_TO_DATE(order_date, '%d-%m-%Y')) AS order_year,
14     MONTH(STR_TO_DATE(order_date, '%d-%m-%Y')) AS order_month,
15     SUM(amount) AS total_revenue,
16     COUNT(DISTINCT order_id) AS total_orders
17 FROM ecommerce_sales
18 GROUP BY
19     YEAR(STR_TO_DATE(order_date, '%d-%m-%Y')),
20     MONTH(STR_TO_DATE(order_date, '%d-%m-%Y'))
21 ORDER BY order_year, order_month;
```

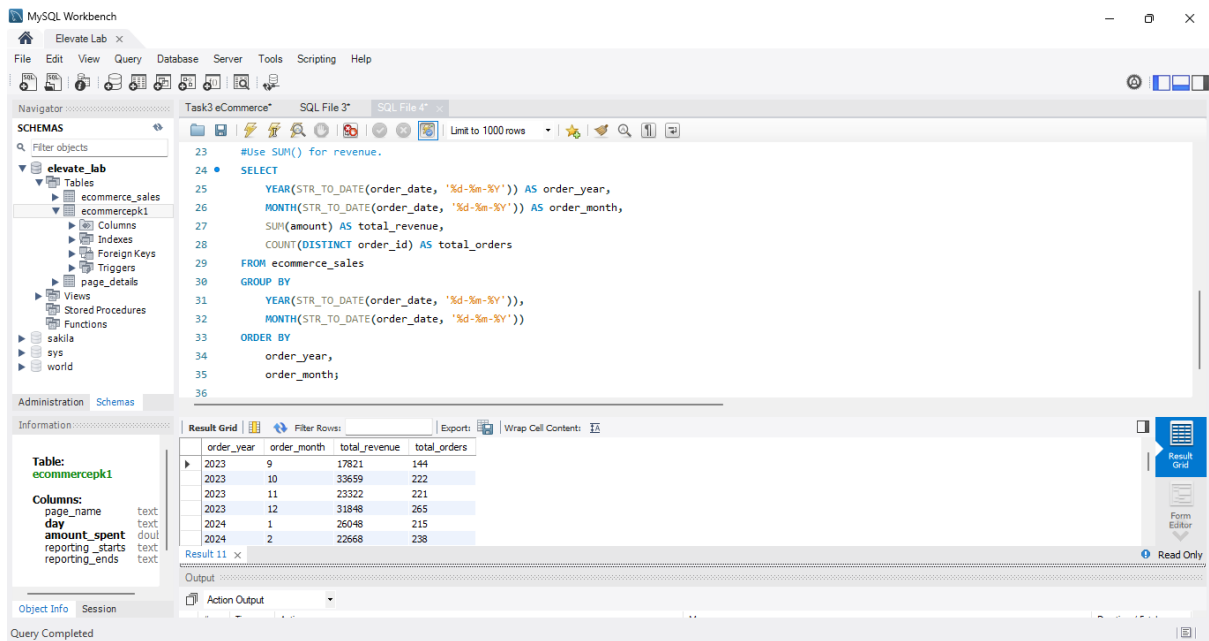
The Result Grid shows the following data:

order_year	order_month	total_revenue	total_orders
2023	9	17821	144
2023	10	33659	222
2023	11	23322	221
2023	12	31848	265
2024	1	26048	215

The Action Output shows the execution of the query:

#	Time	Action	Message	Duration / Fetch
40	14:26:47	SELECT EXTRACT(YEAR FROM order_date) AS order_year, EXTRACT(MONTH FROM...	1 row(s) returned	0.094 sec / 0.000 sec
41	14:28:23	SELECT YEAR(STR_TO_DATE(order_date, '%d-%m-%Y')) AS order_year, MONTH(STR...	25 row(s) returned	0.109 sec / 0.000 sec

C. Use SUM() for revenue.



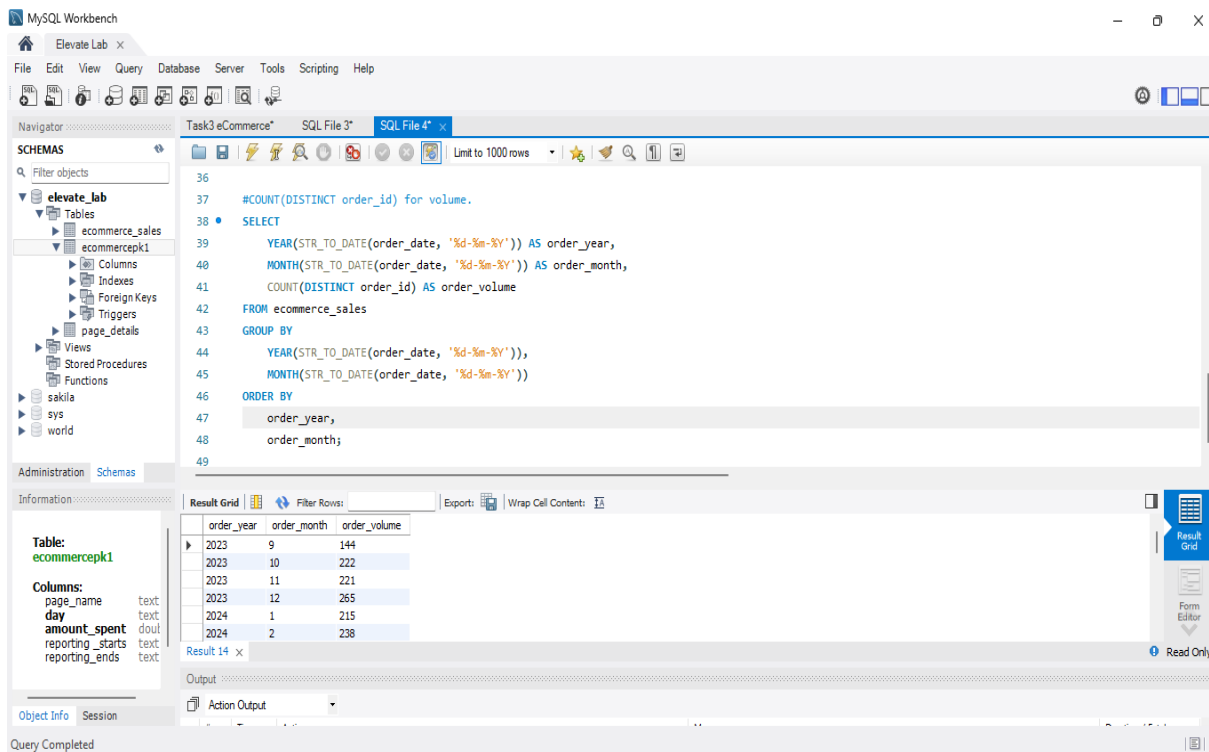
The screenshot shows the MySQL Workbench interface. The SQL editor contains a query to calculate revenue and total orders by year and month. The query is as follows:

```
23 #Use SUM() for revenue.
24 SELECT
25     YEAR(STR_TO_DATE(order_date, '%d-%m-%Y')) AS order_year,
26     MONTH(STR_TO_DATE(order_date, '%d-%m-%Y')) AS order_month,
27     SUM(amount) AS total_revenue,
28     COUNT(DISTINCT order_id) AS total_orders
29 FROM ecommerce_sales
30 GROUP BY
31     YEAR(STR_TO_DATE(order_date, '%d-%m-%Y')),
32     MONTH(STR_TO_DATE(order_date, '%d-%m-%Y'))
33 ORDER BY
34     order_year,
35     order_month;
36
```

The result grid shows the following data:

order_year	order_month	total_revenue	total_orders
2023	9	17821	144
2023	10	33659	222
2023	11	23322	221
2023	12	31848	265
2024	1	26048	215
2024	2	22668	238

D. COUNT(DISTINCT order_id) for volume.



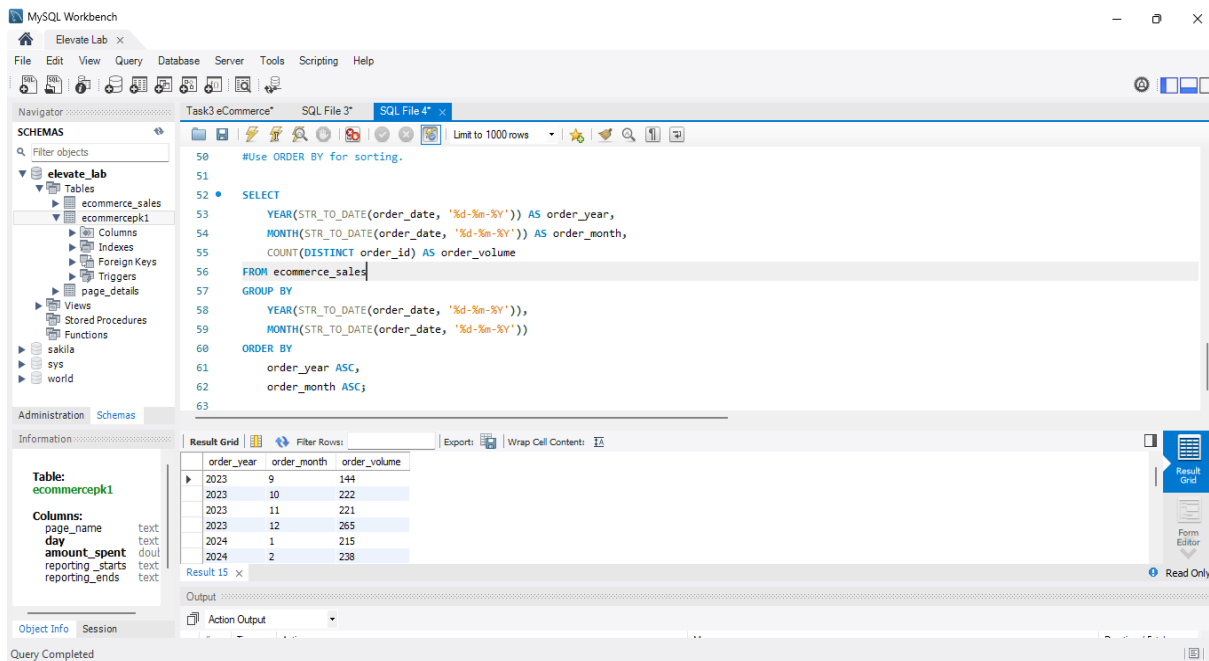
The screenshot shows the MySQL Workbench interface. The SQL editor contains a query to calculate the volume of orders by year and month. The query is as follows:

```
36
37 #COUNT(DISTINCT order_id) for volume.
38 SELECT
39     YEAR(STR_TO_DATE(order_date, '%d-%m-%Y')) AS order_year,
40     MONTH(STR_TO_DATE(order_date, '%d-%m-%Y')) AS order_month,
41     COUNT(DISTINCT order_id) AS order_volume
42 FROM ecommerce_sales
43 GROUP BY
44     YEAR(STR_TO_DATE(order_date, '%d-%m-%Y')),
45     MONTH(STR_TO_DATE(order_date, '%d-%m-%Y'))
46 ORDER BY
47     order_year,
48     order_month;
49
```

The result grid shows the following data:

order_year	order_month	order_volume
2023	9	144
2023	10	222
2023	11	221
2023	12	265
2024	1	215
2024	2	238

E. Use ORDER BY for sorting.



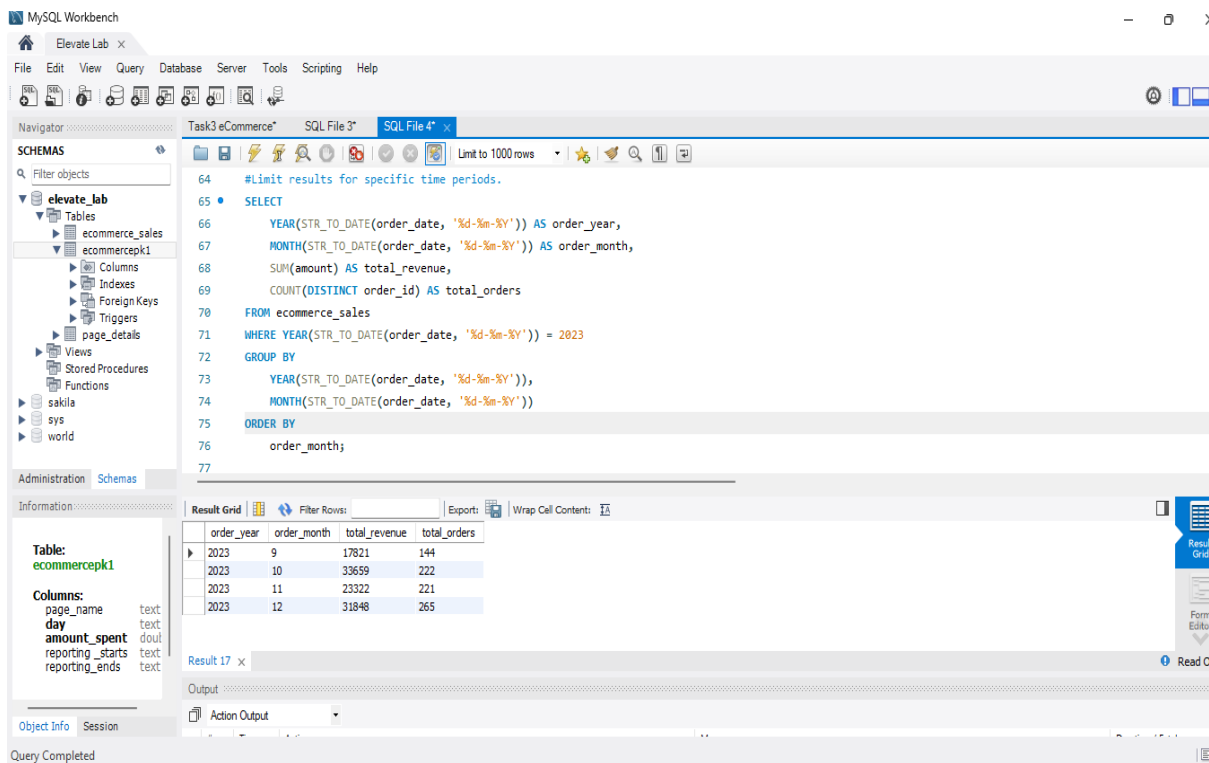
The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'elevator_lab' database schema, including tables like 'ecommerce_sales' and 'ecommercepk1'. The main editor window contains an SQL query that uses the 'ORDER BY' clause to sort results by 'order_year' and 'order_month' in ascending order. The query is as follows:

```
50 #Use ORDER BY for sorting.
51
52 • SELECT
53     YEAR(STR_TO_DATE(order_date, '%d-%m-%Y')) AS order_year,
54     MONTH(STR_TO_DATE(order_date, '%d-%m-%Y')) AS order_month,
55     COUNT(DISTINCT order_id) AS order_volume
56 FROM ecommerce_sales
57 GROUP BY
58     YEAR(STR_TO_DATE(order_date, '%d-%m-%Y')),
59     MONTH(STR_TO_DATE(order_date, '%d-%m-%Y'))
60 ORDER BY
61     order_year ASC,
62     order_month ASC;
63
```

The 'Result Grid' at the bottom displays the following data:

order_year	order_month	order_volume
2023	9	144
2023	10	222
2023	11	221
2023	12	265
2024	1	215
2024	2	238

F. Limit results for specific time periods.



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'elevator_lab' database schema. The main editor window contains an SQL query that filters results for the year 2023 and sorts them by 'order_month' in ascending order. The query is as follows:

```
64 #Limit results for specific time periods.
65 • SELECT
66     YEAR(STR_TO_DATE(order_date, '%d-%m-%Y')) AS order_year,
67     MONTH(STR_TO_DATE(order_date, '%d-%m-%Y')) AS order_month,
68     SUM(amount) AS total_revenue,
69     COUNT(DISTINCT order_id) AS total_orders
70 FROM ecommerce_sales
71 WHERE YEAR(STR_TO_DATE(order_date, '%d-%m-%Y')) = 2023
72 GROUP BY
73     YEAR(STR_TO_DATE(order_date, '%d-%m-%Y')),
74     MONTH(STR_TO_DATE(order_date, '%d-%m-%Y'))
75 ORDER BY
76     order_month;
77
```

The 'Result Grid' at the bottom displays the following data:

order_year	order_month	total_revenue	total_orders
2023	9	17821	144
2023	10	33659	222
2023	11	23322	221
2023	12	31848	265